Day 79: LayerNorm vs BatchNorm — a Real Comparison

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 12, Day 3 | Complete study guide

Watch: Batch Norm vs Layer Norm, Explained (DataMListic) | Layer Normalization in Transformers, Layer Norm Vs Batch Norm (CampusX)

Script: `day79_layernorm_vs_batchnorm.py` · every number in this guide is copied by program from `day79_run_output.txt`, the real output of that script.

LayerNorm normalises each position on its own; BatchNorm normalises each feature over the whole batch. Inside Day 78's encoder block, on Week 9's padded sentences, the two give the same accuracy with batches of 32 (LayerNorm 0.8510, BatchNorm 0.8520). The differences are elsewhere: with BatchNorm a sentence's output changes when the other sentences in its batch change (by up to 1.39e+00), padding changes the output unless it is left out of the statistics (1.38e+00), and at test time the answer depends on whether running averages or batch statistics are used. LayerNorm has none of these, which is why Transformers use it; on this task it did not make the model more accurate, not even with batches of 4 once the learning rate was scaled down.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

Both norms compute a mean and a spread and rescale the numbers by them. **LayerNorm** takes them over the d features of ONE position. **BatchNorm** takes them over every position of every sentence in the batch, for ONE feature at a time; it keeps **running averages** of them during training and uses those at test time, when there may be no batch.

REAL-WORLD SCENARIO

Grading on a curve: LayerNorm compares each student's subjects with their own average; BatchNorm compares each subject's mark with everyone in the room that day, so a student's grade depends on who else sat the exam.

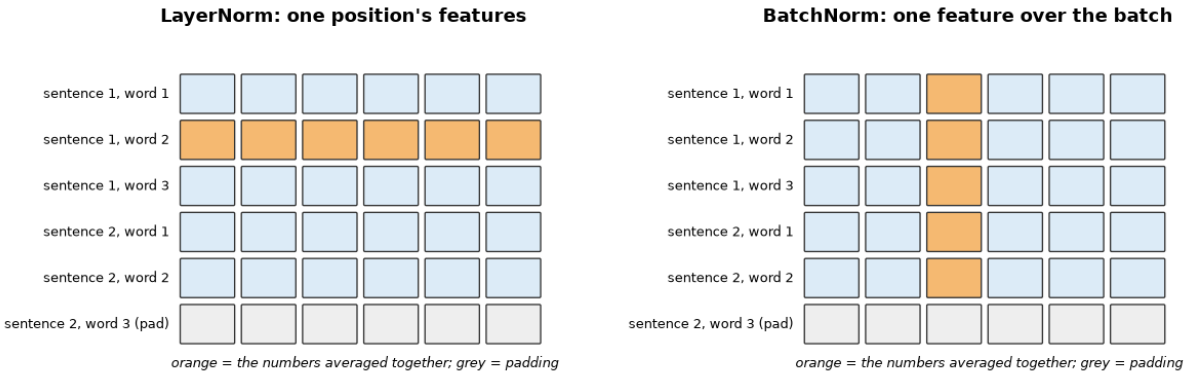
MATHEMATICS

LayerNorm: μ, σ *over the features of one position* (*x has shape sentences × positions × features*)

BatchNorm: μ_f, σ_f *over all real positions of the batch, per feature f; running* $\leftarrow$ *0.9 running + 0.1 batch*

both: $y = (x - \mu) / \sqrt{(\sigma^2 + \varepsilon)} \cdot g + b$

SYSTEM DIAGRAM



Which numbers each norm averages together (orange); grey is padding.

Today's part for Day 83

`day79_exercise.py` is LayerNorm as one function, forward and backward, gradient-checked, beside BatchNorm: Week 12's third part.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. Two ways to normalise the same numbers

DEFINITION — start here, no prior knowledge assumed

The same 2 × 3 × 4 block of numbers through both norms.

REAL-WORLD SCENARIO

The same marks sheet, curved per student or per subject.

IMPLEMENTATION

```
def batch_norm(x, g, b, mask, eps=1e-5):
    real = x[mask]
    mu, var = real.mean(axis=0), real.var(axis=0)
    sd = np.sqrt(var + eps)
    xhat = (x - mu) / sd
    return xhat * g + b, (xhat, sd, mask), mu, var
```

ACTUAL OUTPUT

```
LayerNorm: mean and spread of each position's 4 numbers
  means    0.000   0.000  -0.000  -0.000   0.000   0.000   spreads 1.000 1.000 1.000 1.000 1.000
      1.000
BatchNorm: mean and spread of each feature's 6 numbers
  means   -0.000   0.000  -0.000   0.000   spreads 1.000 1.000 1.000 1.000
BatchNorm's positions are NOT normalised one by one: position means -1.246 -0.929 -0.448 0.150
    0.846 1.627
Actual output from this lesson's run.
```

What it means

After LayerNorm each position has mean 0 and spread 1; after BatchNorm each feature does, but the positions do not: their means run from -1.246 to 1.627. BatchNorm keeps differences between positions and removes differences between features; LayerNorm does the opposite.

WHY CHOOSE THIS?

BatchNorm keeps how positions compare to each other.

WHY NOT THIS? (TRADEOFFS)

It needs a batch to compute anything.

2. Does a sentence's output depend on the rest of its batch?

DEFINITION — start here, no prior knowledge assumed

The first test sentence, put in a batch with 7 other sentences, then with 7 different ones; one untrained block with the same weights for every norm.

REAL-WORLD SCENARIO

Your grade changing because different classmates sat the exam with you.

Why this test

A model whose answer for one input depends on unrelated inputs is hard to test and to serve.

ACTUAL OUTPUT

```
norm                            train mode     eval mode
LayerNorm                         0.00e+00      0.00e+00
BatchNorm (padding left out)      1.39e+00      0.00e+00
BatchNorm (padding counted)       1.23e+00      0.00e+00
Actual output from this lesson's run.
```

What it means

With LayerNorm the companions change nothing (0.00e+00). With BatchNorm in train mode they change the sentence's log-probabilities by up to 1.39e+00. In eval mode BatchNorm uses fixed running averages, so the dependence disappears, but then it computes something different from what it trained with (Section 7).

WHY CHOOSE THIS?

Shows the core difference directly.

WHY NOT THIS? (TRADEOFFS)

Untrained weights; Section 7 measures it on the trained model.

3. Does padding change the output?

DEFINITION — start here, no prior knowledge assumed

The same 8 sentences, padded to their longest and then with 20 more padding positions.

REAL-WORLD SCENARIO

Adding empty chairs to the exam room should not change anyone's grade.

ACTUAL OUTPUT

```
norm                          largest change
LayerNorm                           1.78e-15
BatchNorm (padding left out)        1.78e-15
BatchNorm (padding counted)         1.38e+00
Actual output from this lesson's run.
```

What it means

LayerNorm and BatchNorm with padding left out of the statistics are unaffected (1.78e-15, rounding). BatchNorm that counts padding changes by 1.38e+00: its statistics now depend on how much padding the batch happens to have, which depends on its longest sentence.

WHY CHOOSE THIS?

Leaving padding out fixes it.

WHY NOT THIS? (TRADEOFFS)

BatchNorm then needs the mask everywhere it is used.

4. Gradient check

DEFINITION — start here, no prior knowledge assumed

BatchNorm inside Day 78's tiny tagger, train mode, with and without padding counted, and in a 2-block pre-LN stack.

REAL-WORLD SCENARIO

Checking the new part in place, in every way it is wired.

MATHEMATICS

$$dx_i = (u_i - [i \text{ real}] \, (\text{mean}(u) + z_i \, \text{mean}(u \, z))) / \sigma, \quad u = dy \cdot g, \; z = (x - \mu)/\sigma$$

the means are sums over every position, divided by N, the real positions that made the statistics

IMPLEMENTATION

```
def batch_norm_backward(dy, g, cache):
    xhat, sd, mask = cache
    d, N = xhat.shape[-1], mask.sum()
    dxhat = dy * g
    mean_d = dxhat.reshape(-1, d).sum(axis=0) / N
    mean_dx = (dxhat * xhat).reshape(-1, d).sum(axis=0) / N
    dx = (dxhat - mask[..., None] * (mean_d + xhat * mean_dx)) / sd
    return dx, (dy * xhat).reshape(-1, d).sum(axis=0), dy.reshape(-1, d).sum(axis=0)
```

ACTUAL OUTPUT

```
layer                     : largest |numeric - analytic| = 1.44e-09; every entry passes: True
batch                     : largest |numeric - analytic| = 2.27e-09; every entry passes: True
batch_all                 : largest |numeric - analytic| = 2.36e-09; every entry passes: True
batch, 2 pre-LN blocks    : largest |numeric - analytic| = 2.75e-07; every entry passes: True
Actual output from this lesson's run.
```

What it means

Every entry passes in all four versions.

WHY CHOOSE THIS?

The comparison rests on checked gradients.

WHY NOT THIS? (TRADEOFFS)

Tiny models; the trained ones run the same code.

5. Training with batches of 32

DEFINITION — start here, no prior knowledge assumed

Day 78's one-block tagger and settings, three seeds, with LayerNorm, BatchNorm with padding left out, and BatchNorm with padding counted. BatchNorm is scored with its running averages.

REAL-WORLD SCENARIO

Three curving rules, the same students, the same course.

SPECIFICATION

Training	SGD lr 2.0 (Day 78's choice), clip 1.0, batches of 32, 25 epochs, best epoch on validation.
Same start	Every norm starts from the same weights (Day 78's tagger with the block class as a setting).

Why this test

The claim to test: Transformers use LayerNorm because it trains better.

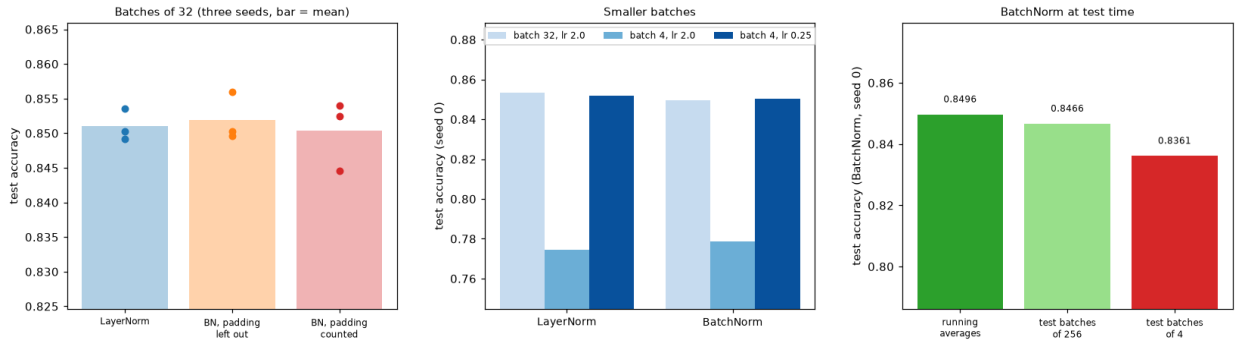
IMPLEMENTATION

```
class NormBlock(EncoderBlock):
    def __init__(self, rng, kind="layer", momentum=0.9, **kw):
        super().__init__(rng, **kw)
        self.kind, self.momentum = kind, momentum
        self.training, self.update_running = True, True
        self.running = {w: (np.zeros(D), np.ones(D)) for w in ("1", "2")}
        self.pos_mask = None
    def _norm(self, x, which):
        if self.kind == "layer":
            return super()._norm(x, which)
        g, b = self.P["g" + which], self.P["b" + which]
        mask = self.pos_mask if self.kind == "batch" else np.ones(x.shape[:2], bool)
        if not self.training:
            mu, var = self.running[which]
            return (x - mu) / np.sqrt(var + 1e-5) * g + b, None
        y, cache, mu, var = batch_norm(x, g, b, mask)
        if self.update_running:
            rm, rv = self.running[which]
            self.running[which] = (self.momentum * rm + (1 - self.momentum) * mu, self.momentum *
                rv + (1 - self.momentum) * var)
        return y, cache
```

```
    def _norm_back(self, dy, which, cache, g):
        if self.kind == "layer":
            return super()._norm_back(dy, which, cache, g)
        dx, g["g" + which], g["b" + which] = batch_norm_backward(dy, self.P["g" + which], cache)
        return dx
```

ACTUAL OUTPUT

```
norm                              seed 0   seed 1   seed 2     mean    train
LayerNorm                         0.8535   0.8502   0.8492   0.8510   0.9542
BatchNorm (padding left out)      0.8496   0.8503   0.8560   0.8520   0.9380
BatchNorm (padding counted)       0.8446   0.8540   0.8525   0.8504   0.9429
Actual output from this lesson's run.
```



Left: batches of 32, three seeds (dots) and their mean (bar). Middle: batches of 4 (Section 6). Right: the trained BatchNorm tagger scored three ways at test time (Section 7).

What it means

All three are the same within the seeds' spread: LayerNorm 0.8510, BatchNorm 0.8520, BatchNorm counting padding 0.8504. The LayerNorm runs reproduce Day 78's exactly (0.8535, 0.8502, 0.8492). On this task, at this batch size, BatchNorm is not worse.

WHY CHOOSE THIS?

A fair test: same weights at the start, same data, three seeds.

WHY NOT THIS? (TRADEOFFS)

One task, one block, one learning rate.

6. Small batches

DEFINITION — start here, no prior knowledge assumed

Batches of 4 sentences: BatchNorm's statistics now come from about 50 words. First at the same learning rate, then at a rate 8 times smaller, so that an epoch moves the weights about as far as with batches of 32.

REAL-WORLD SCENARIO

Curving the grades with only four students in the room.

Why this test

Small batches are where BatchNorm is said to break. The first run, at the same rate, turned out to hurt both norms equally; the second run checks whether that was the learning rate rather than the norm.

ACTUAL OUTPUT

```
norm                            batch 32, lr 2.0  batch 4, lr 2.0  batch 4, lr 0.25
LayerNorm                                 0.8535           0.7746            0.8520
BatchNorm (padding left out)              0.8496           0.7785            0.8503
Actual output from this lesson's run.
```

What it means

At the same learning rate both norms drop, by the same amount: LayerNorm 0.7746, BatchNorm 0.7785. Eight times as many steps of the same size, each from a noisier gradient, is too much for either; it says nothing about BatchNorm.

At a rate 8 times smaller both recover: LayerNorm 0.8520, BatchNorm 0.8503, against 0.8535 and 0.8496 with batches of 32. So BatchNorm with statistics from about 50 words did not break here. Its known trouble with tiny batches would need smaller batches still, or tasks where the statistics matter more, to show; that was not tested.

WHY CHOOSE THIS?

Separates the batch size from the learning rate.

WHY NOT THIS? (TRADEOFFS)

Seed 0 only.

7. BatchNorm at test time

DEFINITION — start here, no prior knowledge assumed

The trained BatchNorm tagger scored with its running averages, and with each test batch's own statistics (batches of 256 and of 4 sentences).

REAL-WORLD SCENARIO

Grading the final exam with the curve from all of last term, or with whoever happens to be in the room.

ACTUAL OUTPUT

```
running averages: 0.8496   test batches of 256: 0.8466   test batches of 4: 0.8361
Actual output from this lesson's run.
```

What it means

The same weights give 0.8496 with running averages, 0.8466 with batches of 256 and 0.8361 with batches of 4. BatchNorm's answer depends on how it is served; LayerNorm's cannot.

WHY CHOOSE THIS?

Running averages make BatchNorm's predictions stable.

WHY NOT THIS? (TRADEOFFS)

They are averages over training batches, not the statistics the model saw at each step.

8. What each costs

ACTUAL OUTPUT

```
LayerNorm 39.8 ms, BatchNorm 40.8 ms per step; ratio 1.03
Actual output from this lesson's run.
```

About the same in this run (ratio 1.03): the normalisation is a small part of a block's work.

Interview preparation — technical questions

Q: Over which axis does each norm compute its statistics?

A: LayerNorm: the features of one position. BatchNorm: one feature over all positions of the batch.

Q: What does BatchNorm do at test time?

A: Uses running averages of its training statistics (momentum 0.9 here).

Q: How should padding be handled in BatchNorm?

A: Left out of the statistics: counted, it changed outputs by 1.38e+00.

Q: Did LayerNorm train better here?

A: No: 0.8510 against 0.8520 with batches of 32, three seeds; and with batches of 4 at a scaled learning rate, 0.8520 against 0.8503.

Q: What changed when the test batch changed?

A: BatchNorm: 0.8496 with running averages, 0.8361 with batches of 4; LayerNorm: nothing.

Interview preparation — theoretical questions

Q: Why do Transformers use LayerNorm?

A: Each position's output depends only on itself: no batch dependence, no padding effects, no train/test mismatch; not, here, higher accuracy.

Q: When is BatchNorm a good choice?

A: Large, fixed-size batches of similar examples, as in CNNs (Day 54).

Q: What does BatchNorm need at generation time, one token at a time?

A: Running averages; batch statistics of one sequence would be meaningless.

Q: Why compare from the same starting weights?

A: So that any difference comes from the norm, not the initialisation.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> On Week 9's sentences, with batches of 32, BatchNorm inside an encoder block is as accurate as LayerNorm. What it is not is independent: its output for one sentence depends on the rest of the batch, on padding unless masked, and on whether running averages or batch statistics are used at test time. LayerNorm's output depends on the position alone, which is what a Transformer that pads, batches and generates one token at a time needs.

- Batches of 32: LayerNorm 0.8510, BatchNorm 0.8520, BatchNorm counting padding 0.8504.
- Batches of 4 (seed 0): 0.7746 / 0.7785 at lr 2.0, 0.8520 / 0.8503 at lr 0.25: the learning rate, not the norm.
- Other sentences in the batch: LayerNorm 0.00e+00, BatchNorm up to 1.39e+00 (train mode).
- 20 extra padding positions: 1.78e-15 unless BatchNorm counts padding (1.38e+00).
- BatchNorm at test time: 0.8496 running averages, 0.8466 batches of 256, 0.8361 batches of 4.

Suggested watch order, if starting from zero

- 1. Batch Norm vs Layer Norm, Explained (DataMListic) — the two norms in five minutes.
- 2. Layer Normalization in Transformers, Layer Norm Vs Batch Norm (CampusX) — a longer walk through why Transformers use LayerNorm.

Day 80 preview

- **The decoder block.** Self-attention that may only look at earlier positions (a causal mask), attention to the encoder's output, and the same add & norm; tested on Week 9's sentences as a next-word task.
- Today's exercise script, `day79_exercise.py`, is LayerNorm as one function: the third part of Day 83's Transformer.